Implementing Expectation Maximisation on Hidden Markov Models with the Baum-Welch Algorithm

Ben Carleton

December 2021

The description and mathematical notation follows closely to Rabiner [1].

The Baum-Welch algorithm is a type of Expectation-Maximisation (EM) algorithm which seeks to estimate the parameters of a Hidden Markov Model (HMM) given a sequence of observed letters $O = O_1 O_2 \dots O_T$ and number of states N. Like all EM algorithms, the Baum-Welch algorithm is an iterative process which converges to a local maximum.

More formally, a HMM's parameters can be defined by $\theta = (A, B, \pi)$ where A represents a matrix of state transitions $A = \{a_{ij}\}$, B represents an emission probabilities distribution, $B = \{b_j(k)\}$ and π represents an initial probabilities distribution, $\pi = \{\pi_i\}$

Initially, we randomly generate $\theta = (A, B, \pi)$. At each iteration, the Baum-Welch algorithm will adjust the model parameters $\theta = (A, B, \pi)$ in order to maximise the probability of the observation sequence, $P(O|\theta)$. Estimating the parameters of the model is a nontrivial task that requires a series of calculations. For each iteration, we first need to calculate the "forward" probabilities α, i.e. the probability of some partial observation sequence, $O_1 O_2 \dots O_t$ up to time t and finishing in s_i. For each state s_i and time-step t we calculate this,

$$\alpha_1(i) = \pi_i b_i(O_1)$$
$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i) a_{ij}\right] b_j(O_{t+1})$$

Whilst this captures the probability that we are in a given state s_i at time t based on observations in the past, we can also calculate the probability that we are in a given state s_i at time t based on observations in the future. We denote these probabilities as "backward" probabilities β and they are calculated using the formulas below:

$$\beta_t(i) = 1$$
$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)$$

Given that both the forward parameter α and the backward parameter β are calculated by an initialisation step and then an inductive step, dynamic programming is used to calculate them in the implementation. Using both the forward and backward probabilities, we can determine the probability that at time t we are in state s_i given both the past and future observations, i.e. the entire observation. This parameter is referred to as γ and calculated as such:

$$\gamma_t(i) = \frac{\alpha_t(i)\beta_t(i)}{\sum_{i=1}^{N} \alpha_t(i)\beta_t(i)}$$

Each γ value is normalized to ensure $\gamma_t(i)$ accurately represents a probability measure, i.e at each time-step the sum of being in any state given an observation and model is 1.

Also using the forward and backward probabilities we can derive the probability that at time t we are in state s_i and at time $t+1$ we are in state s_j. This is shown below,

$$\xi_t(i,j) = \frac{\alpha_t(i)a_{ij}b_j(O_{t+1})\beta_{t+1}(j)}{\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_t(i)a_{ij}b_j(O_{t+1})\beta_{t+1}(j)}$$

Given our new parameters $\alpha_t(i), \beta_t(i), \gamma_t(i)$ and $\xi_t(i,j)$ we can now calculate new model parameters, $\theta = (A, B, \pi)$ based of a maximum likelihood estimate. The new model parameters are defined below:

$$\begin{aligned}\overline{\pi} &= \text{expected frequency in } s_i \text{ at time } 1 \\ &= \gamma_1(i)\end{aligned}$$

$$\begin{aligned}\overline{a}_{ij} &= \frac{\text{expected number of transitions from } s_i \text{ to } s_j}{\text{expected number of transitions from } s_i} \\ &= \frac{\sum_{t=1}^{T-1}\xi_t(i,j)}{\sum_{t=1}^{T-1}\gamma_t(i)}\end{aligned}$$

$$\begin{aligned}\overline{b}_j(k) &= \frac{\text{expected number of times in state } j \text{ and observing } v_k}{\text{expected number of times in state } j} \\ &= \frac{\sum_{t=1 \; s.t. O_t = v_k}^{T}\gamma_t(i)}{\sum_{t=1}^{T}\gamma_t(i)}\end{aligned}$$

This process is then repeated with the new model parameters, $\overline{\theta} = (\overline{A}, \overline{B}, \overline{\pi})$ until convergence is reached. In the implementation, the log likelihood of each iteration is calculated to determine whether convergence is reached. If, after 500 iterations, convergence has not been reached, the implementation will output the 500th iteration as the final result. The MAX_ITERATIONS parameter can be adjusted accordingly.

To account for underflow in the implementation, a scaling coefficient is defined,

$$c_t = \frac{1}{\sum_{i=1}^{N}\alpha_t(i)}$$

$\alpha_t(i)$ and $\beta_t(i)$ are redefined as $c_t\alpha_t(i)$ and $c_t\beta_t(i)$ respectively.

The log likely-hood converges for all test cases. This implies that the probability of each test input sequence occurring increases at each iteration, eventually reaching the point of convergence. Consequently, the implementation is correctly performing expectation maximisation

References

[1] Rabiner, Lawrence. (February 1989) "A tutorial on Hidden Markov Models and Selected Applications in Speech recognition"